

Code Assessment of the MakinaX Smart Contracts

August 19, 2026

Produced for



Contents

1	Executive Summary	3
2	Assessment Overview	4
3	System Overview	5
4	Trust Model	8
5	System Considerations	11
6	Terminology	13
7	Findings	14
8	Limitations and use of report	20

1 Executive Summary

Dear Makina team,

Thank you for trusting us to help Makina with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of MakinaX according to [Scope](#) to support you in forming an opinion on their security risks.

Makina provides MakinaX, a Gnosis Safe module through which authorized operators manage DeFi positions, harvest rewards, swap tokens, and bridge tokens to other chains on behalf of a user who stores their assets in the Safe. The user has to whitelist all operations that can be performed. MakinaX operates in one of three modes: operators are fully trusted in `OPEN` mode, while `FENCED` and `WALLED` add increasing protection against a malicious or compromised operator through slippage limits and cooldowns.

The most critical subjects covered in our audit are escalation of privileges, functional correctness, and trustworthiness.

Security regarding escalation of privileges is good. In a previous iteration, the issues [Manager Can Steal Bridged Assets via Deliberate Non-Fill](#) and [No Rate Limit on Operator Enables Repeated Slippage Loss](#) were identified; both have been addressed in the latest version. Security regarding functional correctness is good after issues such as [Queued Safe Transaction Can Invalidate Accounting Snapshot](#) and [Incoming Token Transfers Corrupt Slippage Check](#) have been addressed. To resolve the latter, Makina changed the specification to ban management instructions that can yield execution control to an attacker, who could exploit it to mask a loss, see [Restrictions on Whitelisted Management Instructions](#). This places considerable responsibility on the user, who must carefully review all management instructions before whitelisting them. Security regarding trustworthiness is good: the trust model is layered and well-defined, with the Safe retaining ultimate control and neither the Provider nor infrastructure roles able to move funds out of the Safe.

The general subjects covered are specification and code complexity. Security regarding both is high.

In summary, we find that the codebase provides a good level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,
ChainSecurity

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the MakinaX repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

Version	Date	Commit Hash	Note
1	18 May 2026	3d6741e9abbd524265433f1eb184bb5a15153d85	Initial Version
2	2 June 2026	7ec1c22dc403c9464bf52964e03278d79f4f12c5	Version with Fixes
3	12 June 2026	97c401835c9ece4c403e9a58ee6f951219260b11	Final Version
4	19 August 2026	2c1338f2c99bc2f5adc51c2104761304115fbf3f	Rename & Permissionless

For the Solidity smart contracts, the compiler version `0.8.35` was chosen.

```
src/
├── MakinaXModule.sol
├── bridge-encoders/
│   ├── AcrossV4BridgeEncoder.sol
│   ├── CctpV2BridgeEncoder.sol
│   └── LayerZeroV2BridgeEncoder.sol
├── factory/
│   └── ModuleFactory.sol
├── flash-loans/
│   └── FlashLoanModule.sol
├── libraries/
│   ├── DecimalsUtils.sol
│   └── Errors.sol
├── module-components/
│   ├── BridgeComponent.sol
│   ├── OracleRegistry.sol
│   ├── SwapComponent.sol
│   └── WeirollComponent.sol
├── registry/
│   └── MakinaXRegistry.sol
└── utils/
    ├── MakinaXContext.sol
    └── MakinaXGovernable.sol
```

2.1.1 Excluded from scope

Any file not explicitly listed in the Scope section is out of scope. In particular, external libraries (e.g. OpenZeppelin), the Safe contracts and any other Safe modules or guards, the protocols used for flash loans, the bridges (Across, CCTP, LayerZero), the price feeds, the Weiroll virtual machine and the off-chain Weiroll instruction generation are not included in the scope.

Additionally, all configurations and operational behavior of the privileged roles are out of scope of this review.

3 System Overview

This system overview describes version (`Version 4`) of the contracts as defined in the [Assessment Overview](#).

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

Makina provides MakinaX, a Gnosis Safe module for strategy operations. Each `MakinaXModule` is attached to one Safe. The Safe holds the assets, and authorized operators act through the module.

Operators can manage positions, harvest rewards, swap tokens, and send tokens to other chains. Position management and harvesting are executed as pre-approved Weiroll instructions in the Safe's context. Swaps and bridge transfers use dedicated module entry points that pull the input token from the Safe before calling the configured external target.

3.1 Instructions

Weiroll instructions are authorized by a Merkle proof against `allowedInstrRoot`, which is set by the Safe. An instruction binds a Weiroll command sequence, position metadata, token lists, an instruction type, and selected state values. A bitmap marks which state values are fixed by the Merkle leaf and which values may be supplied by the operator at execution time.

MakinaX uses four instruction types:

- `MANAGEMENT` changes a position.
- `ACCOUNTING` values a position during management.
- `HARVEST` collects rewards.
- `FLASHLOAN_MANAGEMENT` runs position management in a flash-loan callback.

3.2 Position Management

Operators manage positions by executing `MANAGEMENT` instructions through the Safe. A management call may include an `ACCOUNTING` instruction to value the position before and after the position change.

`WALLED` mode makes this accounting instruction mandatory. In that mode, the module compares the change in position value against the change in the Safe's balances of the management instruction's `affectedTokens`. The comparison applies `maxPositionIncreaseLossBps` or `maxPositionDecreaseLossBps` depending on the position and token flow. `WALLED` mode also enforces a cooldown per management instruction, keyed by the `(positionId, commands, direction)` tuple. Outside `WALLED` mode, the management instruction can run without these checks.

Position management can also use flash loans. A management instruction can request a Morpho flash loan through the shared `FlashLoanModule`. During the Morpho callback, the module executes a nested `FLASHLOAN_MANAGEMENT` instruction and returns the borrowed amount for repayment in the same transaction.

3.3 Harvesting

Operators harvest rewards by executing a `HARVEST` instruction through the Safe. A harvest call can then swap harvested tokens through the same swap path exposed by the standalone swap entry point.

3.4 Swaps

For a swap, the operator selects a Safe-configured swapper and supplies the swap calldata, input amount, and minimum output amount. The module pulls the input token from the Safe, approves the swapper's approval target, calls its execution target, revokes the approval, and measures the received output.

The module charges `swapFeeRate` on the output amount, sends the fee to the registry's fee collector, and returns the remaining output to the Safe. In `FENCED` or `WALLED` mode, the oracle value of the output must also satisfy `maxSwapLossBps` relative to the input, and a swap cooldown is enforced.

3.5 Bridge Transfers

Operators send outbound bridge transfers through bridge encoders stored in `MakinaXRegistry`. The module pulls the input token from the Safe and uses the selected encoder to prepare the external bridge call. The in-scope encoders support Across V4, Circle CCTP V2, and LayerZero V2.

In `FENCED` or `WALLED` mode, the destination recipient must be whitelisted, the transfer's minimum output amount must satisfy the configured loss bound for the selected bridge, and a per-bridge cooldown is enforced. Across additionally requires a registered route. LayerZero additionally requires a registered OFT. The module does not handle inbound bridge messages.

3.6 Pricing

`OracleRegistry` stores one- or two-feed Chainlink routes for tokens and a staleness threshold for each feed. These routes price position values and swap checks. Position values use the configured `accountingCurrency`, or the reference currency when no accounting currency is set.

3.7 Operational States

The module stores two operational flags and an operating mode:

- Paused: a Guardian can pause or unpause the module. Operator entry points require the module not to be paused.
- Suspended: the Provider can suspend or unsuspend the module. Operator entry points require the module not to be suspended.
- Operating mode: the Safe selects one of three modes, ordered by increasing restriction. `OPEN` enforces no additional checks. `FENCED` enforces the swap and bridge checks described above (loss limits, cooldowns, recipient whitelisting, and route/OFT registration). `WALLED` adds the position management checks (mandatory accounting, value preservation, and instruction cooldowns) on top of the `FENCED` restrictions. None of the modes disable operator entry points, and instruction Merkle proof verification is always enforced regardless of mode.

3.8 Deployment and Shared Infrastructure

`ModuleFactory` deploys each MakinaX module as a deterministic minimal proxy using the module implementation stored in `MakinaXRegistry`. Deployment separates two parameter groups: the user-supplied `MakinaXModuleInitParams` (Safe, initial operating mode, allowed instruction root, initial position and swap loss caps, and instruction and swap cooldown durations) and a `MakinaXModuleServiceParams` struct carrying the initial Provider and initial swap fee rate.

The factory exposes two deployment entry points. `createModule` is `restricted` and takes both parameter structs explicitly. `createModuleFree` is permissionless when the governance-controlled `freeDeployment` flag is enabled, and otherwise reverts with `FreeDeploymentDisabled`. It sources the service parameters from the factory's `defaultProvider` and `defaultSwapFeeRate`, and namespaces the deployment salt with the caller (`keccak256(abi.encode(msg.sender, salt))`). The `defaultProvider`, `defaultSwapFeeRate`, and `freeDeployment` values are set by governance through `restricted` setters.

`MakinaXRegistry` is the shared address book for a deployment. It stores the module factory, the module implementation, the fee collector, `FlashLoanModule`, and the bridge encoder for each bridge identifier.

`MakinaXRegistry`, `ModuleFactory`, and the three bridge encoders use OpenZeppelin `AccessManagedUpgradeable` for infrastructure configuration and upgrades.

3.9 Changes from Version 1

The following changes were made in `Version 2` of the codebase:

- **Operating mode:** `Version 1` had no `OperatingMode`. Instead, a single boolean `lockdownMode` flag governed all checks. When enabled, it enforced the swap, bridge, and position management checks together, corresponding to today's `WALLED` mode. When disabled, none of these checks applied, corresponding to today's `OPEN` mode. There was no equivalent of the intermediate `FENCED` mode.
- **Cooldowns:** `Version 1` had no cooldowns or rate limits. The `swapCooldownDuration`, `bridgeCooldownDuration`, and `instrCooldownDuration` parameters, and the per-swap, per-bridge, and per-`(positionId, commands, direction)` cooldowns described above, did not exist.

3.10 Changes from Version 3

The following changes were made in `Version 4` of the codebase:

- **Naming:** `Version 3` used the `MakinaLite` name throughout, including `MakinaLiteModule` and `MakinaLiteRegistry`. The protocol was renamed to `MakinaX`, and the corresponding contracts to `MakinaXModule` and `MakinaXRegistry`.
- **Module service parameters:** `Version 3` did not separate service parameters from init parameters. The `MakinaXModuleServiceParams` struct did not exist, and `initialProvider` and `initialSwapFeeRate` were carried in the module init parameters passed to `ModuleFactory.createModule`. These values are now split into a dedicated struct, and `createModule` takes it explicitly while remaining `restricted`.
- **Permissionless deployment:** `Version 3` had no permissionless deployment path. The `createModuleFree` entry point, the `freeDeployment` flag, and the factory's `defaultProvider` and `defaultSwapFeeRate` parameters described above did not exist; modules could only be deployed through the `restricted` `createModule`.
- **Token recovery:** `Version 3` only exposed `sweepERC20` and `sweepNative`. The Safe-only `sweepERC721` and `sweepERC6909` recovery functions did not exist.

4 Trust Model

The module enforces two layers of access control. The per-module roles (Safe, Provider, Operators, Guardians) are stored as flat state on `MakinaXGovernable` and configured per Safe. The infrastructure contracts (`MakinaXRegistry`, `ModuleFactory`, and the three bridge encoders) are governed by an OpenZeppelin `AccessManager` instance shared across the deployment.

4.1 Roles and Privileges

4.1.1 Safe

The Gnosis Safe to which the module is attached, pinned at initialization on `MakinaXGovernable`. The Safe holds all managed assets.

Trust Level: Fully trusted.

Capabilities: Adds and removes operators and guardians, sets the operating mode (`OPEN`, `FENCED`, or `WALLED`), sets the allowed instruction Merkle root, sets the accounting currency, sets and clears price feed routes and staleness thresholds, sets the position, swap, and bridge value-loss caps, sets cooldown durations, sets per-swapper approval and execution targets, adds and removes whitelisted bridge recipients, and sweeps any stranded balance from the module back to the Safe. The Safe is always a Guardian and cannot be removed from that role.

4.1.2 Provider

The protocol service account, set at initialization and transferable by the current Provider.

Trust Level: Fully trusted.

Capabilities: Sets the swap fee rate. The fee rate can be set up to 100% and is applied to swap output after the minimum output and swap value-loss checks. A Provider can therefore capture the full output of an operator swap or a swap executed after harvest if the fee change is applied before that swap. Suspends and unsuspends the module, which blocks all operator actions while suspended. Transfers the Provider role to a new address. Only the current Provider can transfer the role. The Safe cannot set or replace the Provider.

Suspension does not affect the Safe's own functions. While suspended, the Safe can still configure the module, sweep balances, and manage roles, and can disable the module at the Safe level to recover full control. The Provider cannot block the Safe from accessing funds held by the Safe.

4.1.3 Operators

Addresses authorized by the Safe to call the strategy entry points. Operator actions require the module to be operational (neither paused nor suspended).

Trust Level: Partially trusted in `FENCED` or `WALLED` mode. Fully trusted in `OPEN` mode.

Capabilities: Calls `accountForPosition`, `managePosition`, `harvest`, `swap`, and `sendOutBridgeTransfer` with payloads of their choice. A management instruction can also trigger the nested flash-loan flow that calls `manageFlashLoan`. Operators are always bounded by the Merkle tree of allowed instructions and the swapper targets configured by the Safe. In `FENCED` or `WALLED` mode they are additionally bounded by the swap value-loss cap, the bridge value-loss cap, the bridge recipient whitelist, the bridge encoder route and OFT registrations, and the swap and bridge cooldowns. In `WALLED` mode they are further bounded by the position value-loss caps, mandatory accounting, and instruction cooldowns.

In `OPEN` mode, none of the value-loss caps, the bridge recipient whitelist, or the cooldowns are enforced. Position management runs without accounting and without a loss comparison.

4.1.4 Guardians

Emergency role authorized by the Safe to halt operations. The Safe is permanently a Guardian.

Trust Level: Partially trusted.

Capabilities: Pauses and unpauses the module. While paused, all operator actions revert.

4.1.5 Governance

The address holding the admin role on the `AccessManager` instance governing the MakinaX infrastructure contracts.

Trust Level: Fully trusted.

Capabilities: Grants and revokes `INFRA_CONFIG_ROLE`, `INFRA_UPGRADE_ROLE`, and `STRATEGY_DEPLOYMENT_ROLE`.

4.1.6 Infrastructure Operators

Addresses granted infrastructure roles by Governance.

Trust Level: Fully trusted.

Capabilities:

- `INFRA_CONFIG_ROLE`: changes the registry's stored addresses (module factory, module implementation, fee collector, flash loan module, bridge encoders), sets bridge encoder route and domain or endpoint mappings, and registers or unregisters LayerZero OFT contracts.
- `INFRA_UPGRADE_ROLE`: upgrades the `MakinaXRegistry`, `ModuleFactory`, and bridge encoder implementations behind their respective proxies.
- `STRATEGY_DEPLOYMENT_ROLE`: deploys new module clones via `ModuleFactory`.

The infrastructure roles can affect funds held by existing modules. A module reads the bridge encoder and flash loan module addresses from `MakinaXRegistry` at call time, so `INFRA_CONFIG_ROLE` can repoint a bridge encoder, and `INFRA_UPGRADE_ROLE` can upgrade one in place. During `sendOutBridgeTransfer` the module pulls the input from the Safe to itself and then approves and calls the addresses the encoder returns, so a substituted encoder receives those inputs. This is the reach distinction relative to the Provider and module-level roles: the Provider and Guardian cannot move funds out of the Safe, whereas the infrastructure roles can affect bridge-transfer inputs of existing modules. Existing module clones cannot themselves be re-pointed or upgraded, since they are minimal proxies bound to a fixed implementation.

4.1.7 Fee Collector

The fee collector address registered in `MakinaXRegistry`, set by `INFRA_CONFIG_ROLE`.

Trust Level: Partially trusted.

Capabilities: Receives the fee portion of every swap output. If the fee collector is blacklisted by a token issuer (e.g., USDC, USDT), all swaps whose output is that token will revert across every Safe using the registry.

4.2 External Dependencies

The deployed Safe is assumed to be a stock Gnosis Safe (v1.4.1 or later) with `MakinaXModule` enabled and no additional modules.

Tokens used as instruction inputs, swap inputs, bridge inputs, or oracle quote tokens are assumed to be standard ERC20 contracts with between 6 and 18 decimals, without rebasing, transfer hooks, or fees-on-transfer.

The **accounting currency** sets the granularity of the position value-loss check. It is assumed to be a currency whose oracle price cannot be easily manipulated. Note that rounding in these checks introduces inaccuracies that are more pronounced with high-value & low-decimal token pairs; see [Slippage checks and choice of accounting currency](#).

Price feeds registered through `OracleRegistry` are assumed to implement Chainlink's `AggregatorV2V3Interface` and to price liquid assets that cannot be manipulated within a single transaction. The swap and position value-loss checks in `FENCED` and `WALLED` mode are computed entirely from these feeds, so a manipulable feed weakens the corresponding loss bound.

Accounting instructions registered in `allowedInstrRoot` are assumed to compute position values from manipulation-resistant sources and not to change position state or Safe balances. Their `affectedTokens` list is assumed to include all the relevant tokens. Their output is assumed to report the corresponding position amounts in the same order. The reported position value, and therefore the `WALLED` mode position value-loss check, is derived from this accounting output. An accounting instruction that reads state manipulable within the same transaction (for example, AMM spot reserves) lets the reported value be influenced by the caller.

Management instructions are assumed to spend only tokens listed in their `affectedTokens`, since the loss check sums balances over that list only. Flash-loan management instructions are assumed not to change balances of tokens outside the `affectedTokens` list of their outer management instruction. Harvest instructions are assumed not to spend tokens already held by the Safe. The module does not apply a value-loss or slippage check to the harvest instruction itself. Any swap orders executed after a harvest are checked through the swap path.

Management and flash-loan management instructions are also assumed not to leave persistent Safe approvals after execution.

During bridging, the recipient is delivered the funds on the destination chain, where the source-chain module and Safe have no further control over them. Each whitelisted recipient is assumed to be an address controlled on the destination chain by the same party and able to receive and custody the bridged asset. The Across deposit is encoded with an empty message, so the bridged funds are delivered to the recipient by a plain transfer and no destination-side callback handler is invoked. For Circle CCTP V2 the recipient is the mint recipient, and for LayerZero V2 the recipient receives the OFT token directly.

4.3 Upgradeability

`MakinaXModule` instances are minimal clones produced by `ModuleFactory` via `Clones.cloneDeterministic`. Existing clones cannot be upgraded.

`MakinaXRegistry`, `ModuleFactory`, and the three bridge encoders are deployed behind upgradeable proxies and can be upgraded by `INFRA_UPGRADE_ROLE`.

`FlashLoanModule` is not upgradeable.

5 System Considerations

We leverage this section to highlight findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require a modification inside the project. Instead, they should raise awareness in order to improve the overall understanding for users and developers.

SC1 Bridge Recipients Must Not Be MakinaX Modules

System Consideration	Version 1
Bridge transfer recipients must never be MakinaX module addresses on any chain. If one is configured as a recipient, an attacker who gains execution control mid-swap - for example through a reentrant token pair - can settle a pending inbound transfer into the module, inflating its token balances and causing the slippage check to pass despite the swap destroying value.	

SC2 Price Oracles Must Not Be Configured for Position Tokens

System Consideration	Version 1
<code>SwapComponent._swap</code> uses registered price oracles to bound the slippage of a swap. If a price oracle is configured for a position token - a token representing an underlying position in an external protocol, such as an Aave aToken - the operator can use it as a swap input and the slippage check will pass. However, the check measures only the immediate loss; it is blind to the indirect value destruction. Swapping away an aToken used as collateral for a leveraged position can cause a liquidation in a subsequent block, destroying far more value than the slippage tolerance permits.	

Price oracles must therefore never be registered for position tokens.

SC3 Swap Fees Are Deducted After the Slippage Check

System Consideration	Version 1
In lockdown mode, the slippage check verifies that the value of <code>amountOut</code> is at least <code>inputAmount * (MAX_BPS - maxSwapLossBps) / MAX_BPS</code> . Only after the check passes is the swap fee charged. Hence, the effective maximum loss of a swap is approximately <code>maxSwapLossBps / MAX_BPS + swapFeeRate / MAX_FEE_RATE</code> of the input value.	

SC4 Bridge Adapters Enabled by Default for All Whitelisted Safe Users

System Consideration	Version 1
All bridge adapters registered in the registry are available to every safe by default — there is no per-safe opt-in step to enable a specific adapter. Loss thresholds are initialized to zero for all bridges, which means transfers routed through a suboptimal bridge are bounded by the zero-loss floor. This is intentional by design.	

SC5 Restrictions on Whitelisted Management Instructions

System Consideration

Version 1

The slippage check in `_managePosition` is only robust for management instructions whose execution path is fully determined by the whitelisted script. An instruction that hands control to a third party mid-execution can allow a malicious or compromised operator to circumvent the slippage check, see [Incoming Token Transfers Corrupt Slippage Check](#) for reference.

Examples of instructions that can hand control to a third party include the following:

- Swaps that can route through an attacker-controlled token.
- Callbacks to `tx.origin`, as used by some bridges to refund unused ETH.
- Operator- or user-configurable hooks that can be triggered by management instructions.

SC6 Slippage Checks and Choice of Accounting Currency

System Consideration

Version 1

In `WALLED` mode, `WeirollComponent._managePosition` caps the loss of a management instruction by comparing the position value gained against the value of the affected tokens spent. For a position increase on a non-debt position¹, `_checkPositionMinDelta` requires

$$\text{change} \geq \left\lceil (\text{affectedTokensValueBefore} - \text{affectedTokensValueAfter}) \cdot \frac{\text{MAX_BPS} - \text{maxPositionIncreaseLossBps}}{\text{MAX_BPS}} \right\rceil$$

where `change > 0` is the value gained, `affectedTokensValueBefore - affectedTokensValueAfter > 0` the value spent, and `MAX_BPS = 10000`.

Both sides are priced via `_valueOf`, which floors each token amount to the accounting currency's smallest unit. The affected-token value and the position value are each a sum over individual tokens, so each side can deviate by up to one wei per token. Writing S for the reported value spent and n_a, n_p for the number of affected and position tokens, the realized loss L is then bounded by

$$L < S \cdot \frac{\text{maxPositionIncreaseLossBps}}{\text{MAX_BPS}} + (n_a + n_p) \text{ wei of accounting currency}$$

i.e. the configured relative tolerance plus up to one wei of accounting currency per affected or position token.

This tolerated loss scales with the value of one unit of the accounting currency. With a high-value, low-decimal currency such as WBTC (8 decimals, 100,000 USD), one wei is approximately 0.001 USD, and an operator can mask a significant amount of value across many tokens and repeated calls. The Safe owner, or another party, must therefore implement their own monitoring, tracking the total value of the assets and positions held by the Safe to detect a compromise in time.

¹ For the other cases, a position decrease or a change of a debt position, similar formulas can be derived.

6 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- **Likelihood** represents the likelihood of a finding to be triggered or exploited in practice
- **Impact** specifies the technical and business-related consequences of a finding
- **Severity** is derived based on the likelihood and the impact

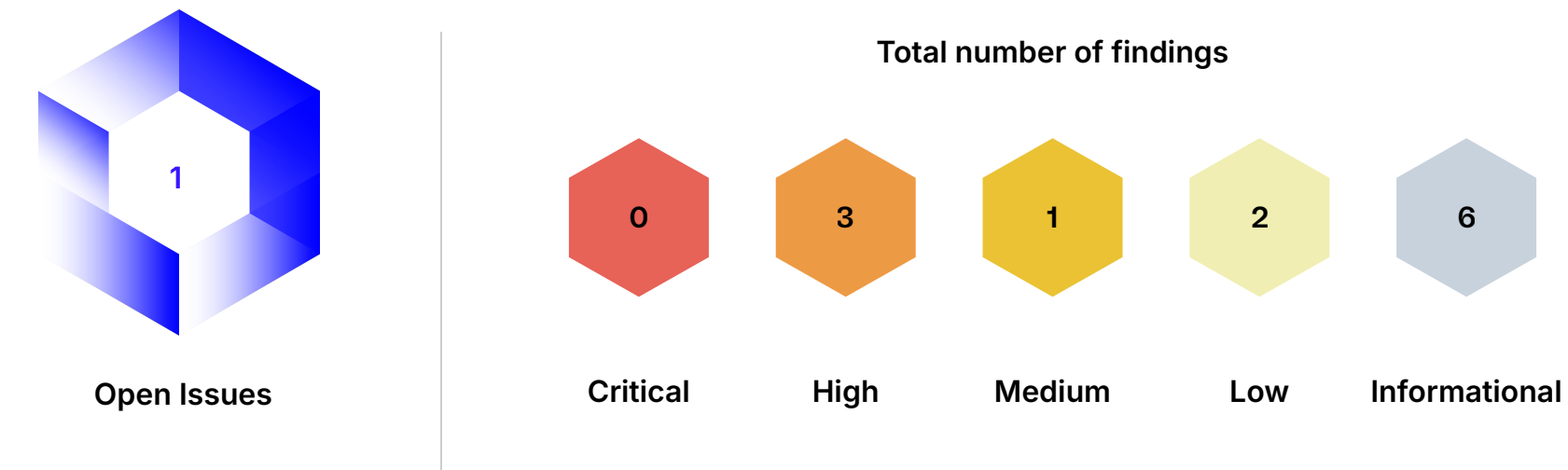
We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

7 Findings

In this section, we describe the findings identified during the course of the engagement. The findings are categorized by severity as explained in the Terminology section. Below we provide an overview of the total number of findings per severity, as well as the number of open issues.



7.1 Overview

The following table lists all identified findings along with their severity and current status. A finding is considered resolved once it has been fully corrected or the specification has been changed accordingly. Non-informational findings with any other status, including partial fixes, acknowledgements, and accepted risks, remain open.

High	#001	Incoming Token Transfers Corrupt Slippage Check	Specification Changed	✓
High	#002	Manager Can Steal Bridged Assets via Deliberate Non-Fill	Code Corrected	✓
High	#003	No Rate Limit on Operator Enables Repeated Slippage Loss	Code Corrected	✓
Medium	#004	Queued Safe Transaction Can Invalidate Accounting Snapshot	Code Corrected	✓
Low	#005	Missing Input Validation of Configuration	Code Corrected	✓
Low	#006	No Upper Bound on minOutputAmount Allows Bridge Griefing	Risk Accepted	⚠
Info	#007	Gas Savings	Code Corrected	✓
Info	#008	Implementation Do Not Disable Initializer	Code Corrected	✓
Info	#009	Inconsistent Use of safeIncreaseAllowance	Code Corrected	✓
Info	#010	Misleading Semantics of "Lockdown Mode"	Code Corrected	✓
Info	#011	Replaced Module Factory Remains Operational	Acknowledged	⊖
Info	#012	Specification Overstates Accounting Instruction Requirement	Specification Changed	✓

7.2 Descriptions

#001 Incoming Token Transfers Corrupt Slippage Check

Design	High	Version 1	Specification Changed ✓
--------	------	-----------	-------------------------

The slippage check in `_managePosition` verifies that the net change in value - accounting for both the position value held by the safe and the aggregate token balances of affected tokens - does not decrease by more than the configured loss tolerance as a result of executing management instructions in `_execute`. Execution can yield control to an attacker, for example via a reentrant token pair in a swap or through callbacks in an integrated protocol.

All three bridging protocols integrated by MakinaX support unpermissioned settlement of completed bridge transfers. Since MakinaX bridges funds between instances on different chains with the safe as the recipient, an attacker who gains execution control mid-instruction can settle a pending inbound bridge transfer, inflating the safe's token balances between the pre- and post-instruction measurements. A loss that would otherwise cause the slippage check to revert is masked by the artificially increased balance. The same applies to cancelled Across transfers that are refunded to the safe: claiming the refund mid-execution similarly inflates balances. More broadly, any protocol the project integrates with that allows permissionless claiming (i.e. reward or yield claiming on behalf of the safe) is subject to the same manipulation.

A similar issue can arise for swaps via malicious token pairs, but only when a MakinaX module is configured as the bridge recipient, which is not the intended design, see [Bridge Recipients Must Not Be MakinaX Modules](#).

Specification Changed

The specification has been changed to not support management instructions that give control over the execution. The Makina states the following:

The loss check is robust for instructions whose execution path is fully determined by the whitelisted script. It cannot be robustly enforced for instructions that embed arbitrary operator-supplied calldata, such as routing through a DEX aggregator. Such calldata can hand control to third parties mid-execution, which can inflate the Safe's measured balances in various ways, thereby masking a real loss. Whitelisting such instructions is bad practice for this reason. A note to this effect was added to the documentation.

#002 Manager Can Steal Bridged Assets via Deliberate Non-Fill

Trust	High	Version 1	Code Corrected ✓
-------	------	-----------	------------------

`AcrossV4BridgeEncoder.getBridgeTransferData` decodes three parameters directly from caller-supplied `order.extraData`: the `refundAddress`, the `outputToken`, and the `fillDeadlineOffset`. While lockdown mode checks that the output route is registered and that `minOutputAmount` meets a floor, it does not restrict `refundAddress` to be an address controlled by the Safe.

A malicious manager can set `refundAddress` to an attacker-controlled address and deliberately send out an unfillable order (e.g., by setting an impossibly high `minOutputAmount` or short `deadline`). Once the fill deadline passes, the Across spoke pool refunds the `inputAmount` to the `refundAddress` chosen by the attacker.

Code Corrected

`AcrossV4BridgeEncoder.getBridgeTransferData` was updated to set the `refundAddress` to the calling module's Safe instead of reading it from caller-supplied data.

#003 No Rate Limit on Operator Enables Repeated Slippage Loss

Design	High	Version 1	Code Corrected ✓
--------	------	-----------	------------------

`MakinaLiteModule` imposes per-operation loss tolerances on swaps, bridging operations and position management instructions but no cooldown or rate limit on the operator. While each individual action must stay within its permitted loss tolerance, a compromised operator can batch as many instructions as the block gas limit allows in a single transaction, accumulating a near complete total loss within a few blocks.

Code Corrected

Configurable cooldown durations have been added to `WeirollComponent`, `SwapComponent`, and `BridgeComponent`, each enforced at a different granularity: management instructions per execution hash (`keccak256(posId, commands, isPositionIncrease)`), swaps globally, and bridge transfers per `bridgeId`.

#004 Queued Safe Transaction Can Invalidate Accounting Snapshot

Security	Medium	Version 1	Code Corrected ✓
----------	--------	-----------	------------------

The slippage check in `_managePosition` evaluates value changes by taking before and after snapshots of position and token value priced using the current `accountingCurrency` and registered price oracles. Execution of `Weiroll` instructions can yield control to an attacker - for example via a reentrant token pair in a swap or through callbacks in an integrated protocol. Once in control, the attacker can execute a queued Safe transaction that has already reached quorum. If such a transaction changes `accountingCurrency` or a registered price oracle, `valueBefore` and `value` are priced under different parameters, rendering the computed `change` meaningless and allowing the slippage bounds to be bypassed.

Code Corrected

The governance setters `setAccountingCurrency` and `setFeedRoute` in `MakinaLiteModule` are now protected by `nonReentrant`, preventing a queued Safe transaction from altering accounting parameters during a management instruction.

#005 Missing Input Validation of Configuration

Design	Low	Version 1	Code Corrected ✓
--------	-----	-----------	------------------

The configuration function `SwapComponent._setSwapperTargets` does not prevent the safe address from being registered as the `executionTarget`. In `_swap`, the execution target is invoked via a low-level `executionTarget.call(order.data)` with fully operator-controlled calldata. If the safe is the target, the operator can use this path to invoke `Safe.execTransactionFromModuleReturnData` with arbitrary data, achieving unrestricted module-level execution on the safe.

Code Corrected

`MakinaLiteModule.setSwapperTargets` now rejects any `executionTarget` or `approvalTarget` equal to the Safe address.

#006 No Upper Bound on minOutputAmount Allows Bridge Griefing

Trust	Low	Version 1	Risk Accepted 
-------	-----	-----------	---

`BridgeComponent._sendOutBridgeTransfer` enforces a lower bound on `order.minOutputAmount` in lockdown mode to prevent slippage, but places no upper bound. A compromised operator can set it to an unsatisfiable value such as `type(uint256).max`, guaranteeing that no solver will fill the order. Once the fill deadline expires the order is cancelled and tokens are refunded, but the attack can be repeated to deny the safe the ability to bridge.

Risk Accepted
Makina accepts this risk but has decided to keep the code unchanged.

#007 Gas Savings

Informational	Version 1	Code Corrected 
---------------	-----------	--

- `BridgeComponent._sendOutBridgeTransfer` loads `_maxBridgeLossBps[order.bridgeId]` from storage on every call, but the value is only consumed inside the `if (lockdownMode)` branch. When `lockdownMode` is false the SLOAD is wasted.

In **Version 2** :

- `WeirollComponent._checkAndSetCooldown` checks `_lastGuardedExecTimestamps[executionHash] != 0` before the cooldown comparison, even though that value is only zero before the first timestamp is set. Reordering the condition so the cooldown comparison is evaluated first lets Solidity short-circuit the common non-reverting path and avoid an extra storage read.

Code Corrected
Both gas savings have been adopted:

- `BridgeComponent._sendOutBridgeTransfer` now loads `_maxBridgeLossBps` only within the guarded branch.
- `WeirollComponent._checkAndSetCooldown` (and equivalent checks in `SwapComponent` and `BridgeComponent`) evaluate the cooldown first, making the common non-reverting path cheaper.

#008 Implementation Do Not Disable Initializer

Informational	Version 1	Code Corrected 
---------------	-----------	--

Neither `ModuleFactory` nor `MakinaLiteRegistry` calls `_disableInitializers` in their constructors. For upgradeable contracts, calling `_disableInitializers` in the implementation constructor is considered best practice to prevent the implementation itself from being initialized.

Code Corrected
Both `ModuleFactory` and `MakinaLiteRegistry` now call `_disableInitializers()` in their constructors.

#009 Inconsistent Use of safeIncreaseAllowance

Informational

Version 1

Code Corrected ✓

`FlashLoanModule._handleFlashLoanCallback` uses `safeIncreaseAllowance` to grant the module an allowance to pull flash-loaned tokens, while the rest of the codebase uses `forceApprove`. Beyond the inconsistency, `safeIncreaseAllowance` interacts poorly with tokens that implement ERC-7674 (transient allowances): `allowance()` under ERC-7674 returns the sum of persistent and temporary allowances, so `safeIncreaseAllowance` reads that combined value and sets a new persistent allowance equal to it plus the requested amount, silently converting any outstanding temporary allowance into a permanent one. No temporary allowances are granted in this context, so there is no immediate impact.

Code Corrected

`FlashLoanModule` now uses `forceApprove` throughout.

#010 Misleading Semantics of "Lockdown Mode"

Informational

Version 1

Code Corrected ✓

The abstract contract `MakinaLiteGovernable`, inherited by `MakinaLiteModule`, exposes a boolean `lockdownMode` that gates the contract's core security guarantees: in lockdown mode, operators are restricted to whitelisted bridge recipients, mandatory accounting instructions, and bounded value loss per operation. The naming implies that lockdown is a special, restrictive state activated only in exceptional circumstances.

In practice the inverse is true: outside lockdown mode the operator is fully trusted, can destroy arbitrary value, and can bridge tokens to any address. The Merkle-tree allowlist for instructions remains in effect but provides no financial protection. Users who treat lockdown mode as an optional extra layer of security on top of an already-safe baseline are likely to underestimate the risk of deploying without it.

Code Corrected

The boolean `lockdownMode` in `MakinaLiteGovernable` has been replaced with a three-level `OperatingMode` enum (`OPEN`, `FENCED`, `WALLED`), where each level is documented to reflect progressively stricter operational constraints.

#011 Replaced Module Factory Remains Operational

Informational

Version 1

Acknowledged ⚡

`ModuleFactory.createModule` does not verify that it is still the factory registered in `MakinaLiteRegistry`. When `setModuleFactory` replaces the factory, the old one continues to accept deployments, cloning whatever `moduleImplementation` the registry currently points to — including implementations introduced after its replacement. Modules deployed via a deprecated factory are tracked only in that factory's own `isMakinaLiteModule` mapping, making them invisible to the registry and to any integrator relying on it.

Acknowledged

Makina has decided to keep the code unchanged.

#012 Specification Overstates Accounting Instruction Requirement

Informational

Version 1

Specification Changed 

The specification states: "A `MANAGEMENT` instruction is always paired with an `ACCOUNTING` instruction to account for the changes it introduces. In lockdown mode, the accounting instruction is mandatory." The second sentence correctly scopes the requirement to lockdown mode, but the first sentence - "always paired" - is incorrect since the accounting instruction is optional when `lockdownMode` is false.

Specification Changed

The `IWeirollComponent` NatSpec now explicitly documents `acctInstruction` as optional outside of `WALLED` mode, correctly reflecting that value preservation checks are only enforced when guarded.

8 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.